STOCKWHISK SAAS PLATFORM

DOC-01

Software Requirements Specification (SRS)

IEEE 830 Standard Functional & Non-Functional Requirements Specification

DOCUMENT TYPE	TARGET AUDIENCE	RELEASE VERSION	STATUS
Technical Architecture	Core Engineering, QA, System Architects	v1.4.0 Production	✅ Verified & Approved

Software Requirements Specification (SRS) for StockWhisk

1. Introduction

1.1 Purpose

The purpose of this document is to define the requirements for StockWhisk, a multi-tenant SaaS Inventory, Point of Sale (POS), Warranty, and Financial Management system designed primarily for retail and service businesses in Bangladesh.

1.2 Scope

The StockWhisk platform encompasses the following primary features:
- **Inventory Tracking:** Real-time tracking of stock levels, including per-unit serial lifecycle tracking.
- **Point-of-Sale (POS):** Fast and efficient checkout with barcode scanning, multiple payment methods, and EMI scheduling.
- **Customer & Supplier Management (CRM):** Tracking customer profiles, due balances, supplier accounts, and purchase histories.
- **Warranty & Service Repairs:** Managing warranty claims, verifying warranty status, and tracking repair service tickets.
- **Financial Accounting:** Ledger-based accounting for cash flows, expenses, daily settlements, and investment tracking.
- **Multi-branch Operations:** Stock transfers and branch-level reporting.
- **Platform Administration:** Billing, subscription management, tenant onboarding, and platform configurations.

1.3 Tech Stack

- **Backend:** Django REST Framework (Python 3.x)
- **Frontend:** Next.js 14 (React, TypeScript, Tailwind CSS)
- **Database:** PostgreSQL (production), SQLite/MySQL support
- **Async Tasks:** Celery + Redis for background processing
- **Deployment:** Docker, Docker Compose (with Caddy reverse proxy)

2. Overall Description

2.1 Product Perspective

StockWhisk is a cloud-hosted Software as a Service (SaaS). It employs strict tenant isolation at the application level using a thread-local context and a `TenantScopedModel` base class. This ensures data security and privacy for each shop without requiring separate database schemas, simplifying maintenance and scaling.

2.2 User Classes

- **Platform Super Admin:** Platform owner/operator managing subscriptions, tenants, and system-wide settings.
- **Shop Owner:** Primary account holder for a tenant, with full access to their shop's data and billing.
- **Shop Manager:** An employee with broad permissions configured by the Shop Owner to oversee daily operations.
- **Cashier:** A frontline staff member primarily interacting with the POS, sales, and basic customer operations.
- **Reseller:** A third-party partner who refers new shops to StockWhisk and tracks commissions via a dedicated portal.

2.3 Operating Environment

- **Client Side:** Modern web browsers (Chrome, Firefox, Safari, Edge). Responsive design for desktop, tablet, and mobile viewing.
- **Server Side:** VPS deployment using Docker Compose. Components include a Caddy reverse proxy for SSL, the Django backend application, Celery workers/beat for scheduled tasks, and the PostgreSQL database.

2.4 Design Constraints

- **Multi-tenant Data Isolation:** Must strictly enforce tenant isolation within a single database using `tenant_id` foreign keys and thread-local context filtering.
- **Auditability:** Financial data (LedgerEntry) must be append-only to maintain strict audit trails and prevent historical data tampering.
- **Authentication:** Stateless JWT (JSON Web Tokens) for API authentication.

3. Functional Requirements

3.1 Authentication & Authorization (FR-AUTH)

- **Registration:** Phone number or email registration with OTP verification.
- **Login:** Secure JWT-based login.
- **Password Reset:** OTP or email link based password recovery.
- **Permissions:** Role-based access control (RBAC) governing access to specific modules and actions.

3.2 Catalog Management (FR-CATALOG)

- **Product CRUD:** Create, read, update, delete products.
- **Categorization:** Manage categories, brands, and units of measurement.
- **Serial Tracking:** Manage `ProductUnit` items for individual serial numbers.
- **Inventory Toggle:** `track_inventory` boolean to differentiate between physical goods and services.
- **Barcodes:** Automatic or manual barcode generation for products.

3.3 Point of Sale (FR-POS)

- **Interface:** Fast, keyboard-friendly interface for cart management.
- **Scanning:** Barcode scanner integration for rapid item addition.
- **Payments:** Multi-payment checkout support (Cash, bKash, Nagad, Card, Bank).
- **Quotation:** Ability to save carts as quotations without deducting stock.
- **EMI:** Direct integration for setting up EMI schedules from the checkout screen.

3.4 Sales Management (FR-SALES)

- **Invoices:** Listing, viewing, and generating PDF invoices for completed sales.
- **Sharing:** Direct WhatsApp sharing of invoices.
- **Payments:** Adding subsequent payments to open or partially paid invoices.
- **Corrections:** Mechanisms for authorized users to correct sales records (typically via returns/adjustments).

3.5 Returns (FR-RETURNS)

- **Processing:** Handling sale returns with options to restock items.
- **Refunds:** Processing cash or digital refunds.
- **Exchanges:** Managing product replacements within the return workflow.

3.6 Inventory Management (FR-INVENTORY)

- **Ledger:** Detailed `StockMovement` ledger tracking all inventory changes.
- **Adjustments:** Manual stock adjustments (loss, damage, audit corrections).
- **Alerts:** Low-stock notifications based on configurable thresholds.
- **Lifecycle:** Tracking the state of individual serialized units (IN_STOCK → SOLD → DEFECTIVE → RETURNED_SUPPLIER).

3.7 Purchasing (FR-PURCHASING)

- **Suppliers:** Managing supplier profiles and contact information.
- **Purchase Orders:** Creating POs for stock replenishment.
- **Receiving:** Processing received POs to increment stock levels.
- **Payments:** Tracking payments to suppliers, including credit purchases and outstanding balances.

3.8 Customer Relationship Management (FR-CRM)

- **Profiles:** Maintaining customer details and purchase history.
- **Due Balances:** Tracking outstanding due amounts for credit sales.
- **Collections:** Processing due collections seamlessly without artificially inflating total sales revenue.
- **History:** Comprehensive view of all customer interactions and payments.

3.9 EMI Management (FR-EMI)

- **Scheduling:** Creating EMI schedules based on down payment, interest rate, and duration.
- Formula: Principal = Total - DownPayment; Interest = Principal × Rate/100; Monthly = (Principal+Interest)/Months
- **Tracking:** Recording and tracking individual installment payments and statuses.

3.10 Accounting & Finance (FR-ACCOUNTING)

- **Expenses:** Tracking operational expenses categorized by custom `ExpenseCategory`.
- **Ledger:** Strict, append-only `LedgerEntry` for all cash flow tracking.
- **Settlements:** `DailySettlement` for shift closings, including denomination counting.
- **Investments:** Tracking capital additions and drawings.
- **Transfers:** Inter-account fund movements (e.g., Cash to Bank).
- **Reporting:** Profit & Loss (P&L) statements and Financial Position reporting.

3.11 Warranty & Service (FR-SERVICE)

- **Verification:** Checking warranty status via serial number or invoice (ACTIVE / EXPIRING_SOON / EXPIRED).
- **Ticketing:** Managing repair service tickets with states (RECEIVED → DIAGNOSING → AWAITING_PARTS → IN_REPAIR → READY → DELIVERED / CANCELLED).
- **Parts:** Tracking parts used in repairs, with optional inventory stock deduction.
- **Claims:** Managing claims made to suppliers or manufacturers.

3.12 Reports & Analytics (FR-REPORTS)

- **Dashboards:** HIGH-level sales and profit overviews.
- **Analytics:** Dead stock analysis, top-performing products.
- **Filtering:** Comprehensive date-range and criteria filtering.
- **Exports:** Exporting report data in CSV and PDF formats.

3.13 Settings & Configuration (FR-SETTINGS)

- **Profile:** Shop details, logo, address, and VAT/Tax configurations.
- **Toggles:** Feature toggles to enable/disable modules (e.g., hiding EMI if not used).
- **Staff Management:** Inviting users and assigning roles/permissions.

3.14 Billing & Subscriptions (FR-BILLING)

- **Plans:** Managing SaaS subscription tiers and features.
- **Payments:** Uploading manual payment proofs (e.g., bKash TRX ID, Bank deposit slips) for admin verification.
- **Trials:** Managing free trial periods and expirations.

3.15 Notifications (FR-NOTIFICATIONS)

- **In-App:** Real-time alerts for system events.
- **Digests:** Automated low-stock email/SMS digests.
- **Reminders:** Automated EMI and due payment reminders.
- **WhatsApp:** Configuration for automated WhatsApp messaging.

3.16 Multi-branch Operations (FR-BRANCHES)

- **Transfers:** Initiating and receiving stock transfers between different physical branch locations.

3.17 Reseller Portal (FR-RESELLERS)

- **Tracking:** Reseller portal for tracking referred shops, subscription payments, and calculating commissions.

4. Non-Functional Requirements

- **Performance:** API endpoints must respond within 200ms at the 95th percentile. POS operations must feel instantaneous to the cashier.
- **Security:** HTTPS everywhere. Strict tenant isolation. Protection against common web vulnerabilities (XSS, CSRF, SQLi via ORM).
- **Scalability:** Stateless backend design allows horizontal scaling of Django application servers.
- **Availability:** Target 99.9% uptime. Automated daily database backups.
- **Usability:** Intuitive UX tailored for high-speed retail environments.
- **Internationalization (i18n):** Full support for English and Bangla interfaces and messaging.

5. External Interface Requirements

- **User Interfaces:** Web-based, responsive UI built with Next.js and Tailwind CSS.
- **API Interfaces:** RESTful JSON APIs documented via Swagger/OpenAPI for potential third-party integrations.
- **Hardware Interfaces:** Support for standard USB/Bluetooth barcode scanners (acting as keyboard input) and thermal receipt printers (via browser print dialog or specialized print drivers).